2650026_lab4

November 2, 2026

Roll No.	Subject Code	Assignment No.
2650026	CS69101	Lab 4

Given a set of integer elements stored in an input file, `input.txt`, construct a Red-Black Tree by inserting the elements one by one. Implement insertion and deletion operations while maintaining all the Red-Black Tree properties. Store the final tree contents in `output.txt` using in-order traversal.

[12]:
```
import sys
from io import TextIOWrapper
from typing import Self
import random
import sys
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.pylab import figure

sys.setrecursionlimit(100000) # Fix to avoid RecursionError

prep_arr = [random.randint(1, (i+1)*10) for i in range(5000)] # Random Array of␣
 ↪5000 Elements
# prep_arr.sort() # Sorted Array for Worst Case Scenario
print(prep_arr[:25])
print(len(prep_arr))

# Write the array to a file
f = open("./2650026_lab4_data/input0.txt", "w")
for i in prep_arr:
    f.write(f"{i}, ")
f.write("\n")
f.close()

del prep_arr
del f

iterations = 0
PRINT_DEBUG = False
```

```
[6, 16, 22, 24, 46, 33, 53, 26, 11, 39, 98, 15, 19, 90, 146, 130, 87, 66, 89,
59, 142, 41, 69, 9, 168]
5000
```

1 Q1 & Q2

Read integer contents from `input.txt` and construct a Red-Black Tree using the `insert(key)` operation.

Implement Red-Black Tree insertion, including the required recolouring and LL, RR, LR, and RL rotations to maintain the Red-Black Tree properties.

1.1 File Read

[13]:
```
# Read a set of integer elements from an input file (input.txt)
arr0 = []
data0 = open("./2650026_lab4_data/input.txt", 'r')

try:
    for line in data0.readlines():
        line_data = line.rstrip().rstrip(',').replace(' ','').split(',') #using␣
 ↪rstrip to remove the \n
        for data in line_data:
            arr0.append(int(data))
    data0.close()
except Exception as e:
    sys.exit("[ERROR] Invalid data" + str(e))

print(f"Read {len(arr0)} elements.")
```

```
Read 17 elements.
```

[14]:
```
print(arr0)
```

```
[70, 50, 90, 77, 55, 348, 76, 67, 69, 420, 25, 911, 210, 108, 768, 1080, 720]
```

1.2 Red Black Tree

[15]:
```
class RB_Node:
    def __init__(self, value: int | None = None) -> None:
        self.__value = value
        self.__lchild = None
        self.__rchild = None
        self.__parent = None
        self.__color = '#F00' # Color in #RGB, new node is always red

    def __get_lchild(self) -> Self | None:
        return self.__lchild
```

```
    def __set_lchild(self, lchild: Self | None) -> None: # type: ignore
        if PRINT_DEBUG:
            print(f"Setting left child of node with value {self.__value} to␣
↪{lchild}")
        self.__lchild = lchild
        if lchild is not None:
            lchild.__parent = self
            # lchild.update_height_and_balance()  # Update the height and␣
↪balance factor of the left child when it is set
        # self.update_height_and_balance()  # Update the height and balance␣
↪factor of the current node when the left child is set

    lchild = property(__get_lchild, __set_lchild, doc="Left child of the node")

    def __get_rchild(self) -> Self | None:
        return self.__rchild

    def __set_rchild(self, rchild: Self | None) -> None: # type: ignore
        if PRINT_DEBUG:
            print(f"Setting right child of node with value {self.__value} to␣
↪{rchild}")
        self.__rchild = rchild
        if rchild is not None:
            rchild.__parent = self
            # rchild.update_height_and_balance()  # Update the height and␣
↪balance factor of the right child when it is set
        # self.update_height_and_balance()  # Update the height and balance␣
↪factor of the current node when the right child is set

    rchild = property(__get_rchild, __set_rchild, doc="Right child of the node")

    def __get_parent(self) -> Self | None:
        return self.__parent

    def __set_parent(self, parent: Self | None) -> None: # type: ignore
        if PRINT_DEBUG:
            print(f"Setting parent of node with value {self.__value} to␣
↪{parent}")
        self.__parent = parent
        if parent is not None:
            # parent.update_height_and_balance()  # Update the height and␣
↪balance factor of the parent when it is set
            pass

    parent = property(__get_parent, __set_parent, doc="Parent of the node")
```

```
    def __get_value(self) -> int | None:
        return self.__value

    def __set_value(self, value: int) -> None:
        self.__value = value

    value = property(__get_value, __set_value, doc="Value of the node")

    def __get_color(self) -> str | None:
        return self.__color

    def __set_color(self, color: str) -> None:
        self.__color = color

    color = property(__get_color, __set_color, doc="Color of the node")

    def __str__(self) -> str:
        return f"Node(value={self.__value}, color={self.__color}, lchild={self.
 ↪__lchild.value if self.__lchild else None}, rchild={self.__rchild.value if␣
 ↪self.__rchild else None}, parent={self.__parent.value if self.__parent else␣
 ↪None})"

    def __repr__(self) -> str:
        return self.__str__()
```

[16]:
```
def RB_right_rotate(k2_node: RB_Node | None, root_node: RB_Node | None) ->␣
 ↪RB_Node | None:
    global iterations
    if PRINT_DEBUG:
        print(f"Right Rotate: k2_node={k2_node}, root_node={root_node}")
    k1_node = k2_node.lchild if k2_node is not None else None
    if k1_node is None or k2_node is None:
        return root_node

    iterations += 1
    if PRINT_DEBUG:
        print(f"LL Case: k1_node={k1_node}, k2_node={k2_node}")

    # From Weiss' Data Structures and Algorithm Analysis in C++ (Page 156)
    # Combined CLRS Logic with Weiss' Logic to avoid bugs

    # SAFETY CODE
    TEMP_NODE = k1_node.rchild
    OG_PARENT = k2_node.parent
    WAS_RCHILD = bool(OG_PARENT is not None and OG_PARENT.rchild is k2_node)

    # CLRS: x.lchild = y.rchild
```

```
    # CLRS: y.rchild = x
    k1_node.rchild = k2_node
    k2_node.lchild = TEMP_NODE

    # Update Parents

    # CLRS: if y.rchild is not None:
    # CLRS:     y.rchild.parent = y
    # CLRS: // However, y.rchild is now x.lchild, so we need to update x.
↪lchild's parent to y
    if PRINT_DEBUG:
        print(f"Updating parent of {k2_node.lchild} to {k2_node}")
    if k2_node.lchild is not None:
        k2_node.lchild.parent = k2_node

    # CLRS: x.parent = y.parent
    k1_node.parent = OG_PARENT

    # CLRS: if y.parent is None:
    # CLRS:     root_node = y
    if OG_PARENT is None:
        root_node = k1_node
    # CLRS: elseif y is y.parent.rchild:
    # CLRS:     y.parent.rchild = x
    elif WAS_RCHILD:
        OG_PARENT.rchild = k1_node
        # OG_PARENT.update_height_and_balance() # Update height and balance␣
↪factor of OG_PARENT
    # CLRS: else:
    # CLRS:     y.parent.lchild = x
    else:
        OG_PARENT.lchild = k1_node
        # OG_PARENT.update_height_and_balance() # Update height and balance␣
↪factor of OG_PARENT


    # CLRS: y.parent = x
    # k2_node.parent = k1_node (THIS IS AUTO HANDLED BY SETTER METHOD)

    # if k1_node.rchild is not None:
    #     k1_node.rchild.parent = k1_node

    # root_node = k1_node if OG_PARENT is None else root_node // Copilot told␣
↪me this

    # k1_node.update_height_and_balance() # Update height and balance factor of␣
↪k1_node
```

```
    # k2_node.update_height_and_balance() # Update height and balance factor of␣
↪k2_node

    if k1_node.parent is not None:
        # k1_node.parent.update_height_and_balance() # Update height and␣
↪balance factor of k1_node's parent
        pass

    return root_node
```

[17]:
```
def RB_left_rotate(k1_node: RB_Node | None, root_node: RB_Node | None) ->␣
↪RB_Node | None:
    global iterations
    if PRINT_DEBUG:
        print(f"Left Rotate: k1_node={k1_node}, root_node={root_node}")
    k2_node = k1_node.rchild if k1_node is not None else None
    if k1_node is None or k2_node is None:
        return root_node

    iterations += 1
    if PRINT_DEBUG:
        print(f"RR Case: k1_node={k1_node}, k2_node={k2_node}")

    # From Weiss' Data Structures and Algorithm Analysis in C++ (Page 156)
    # Combined CLRS Logic with Weiss' Logic to avoid bugs

    # SAFETY CODE
    TEMP_NODE = k2_node.lchild
    OG_PARENT = k1_node.parent
    WAS_RCHILD = bool(OG_PARENT is not None and OG_PARENT.rchild is k1_node)

    # CLRS: x.rchild = y.lchild
    # CLRS: y.lchild = x
    k2_node.lchild = k1_node
    k1_node.rchild = TEMP_NODE

    # Update Parents

    # CLRS: if y.lchild is not None:
    # CLRS:     y.lchild.parent = x
    # CLRS: // However, y.lchild is now x.rchild, so we need to update x.
↪rchild's parent to y
    if k1_node.rchild is not None:
        k1_node.rchild.parent = k1_node

    # CLRS: y.parent = x.parent
    k2_node.parent = OG_PARENT
```

```
    # CLRS: if x.parent is None:
    # CLRS:     root_node = y
    if OG_PARENT is None:
        root_node = k2_node
    # CLRS: elseif x is x.parent.lchild:
    # CLRS:     x.parent.lchild = y
    elif WAS_RCHILD:
        OG_PARENT.rchild = k2_node
        # OG_PARENT.update_height_and_balance() # Update height and balance␣
↪factor of OG_PARENT
    # CLRS: else:
    # CLRS:     x.parent.rchild = y
    else:
        OG_PARENT.lchild = k2_node
        # OG_PARENT.update_height_and_balance() # Update height and balance␣
↪factor of OG_PARENT

    # CLRS: x.parent = y
    # k1_node.parent = k2_node (THIS IS AUTO HANDLED BY SETTER METHOD)

    # k1_node.update_height_and_balance() # Update height and balance factor of␣
↪k1_node
    # k2_node.update_height_and_balance() # Update height and balance factor of␣
↪k2_node

    if k2_node.parent is not None:
        # k2_node.parent.update_height_and_balance() # Update height and␣
↪balance factor of k2_node's parent
        pass

    return root_node
```

[18]:
```
#   k3
#   /
# k1          =>        k2
#  \                    / \
#   k2                k1 k3

def RB_double_rotate_right(k3_node: RB_Node | None, root_node: RB_Node | None)␣
↪-> RB_Node | None:
    if PRINT_DEBUG:
        print(f"Double Right Rotate: k3_node={k3_node}, root_node={root_node}")
    k1_node = k3_node.lchild if k3_node is not None else None
    if k1_node is None or k3_node is None:
        return root_node
```

```
    if PRINT_DEBUG:
        print(f"Double Right Rotate: k1_node={k1_node}, k3_node={k3_node}")

    root_node = RB_left_rotate(k1_node, root_node)
    root_node = RB_right_rotate(k3_node, root_node)

    # k3_node.update_height_and_balance() # Update height and balance factor of␣
↪k3_node
    # k1_node.update_height_and_balance() # Update height and balance factor of␣
↪k1_node

    if k3_node.parent is not None:
        # k3_node.parent.update_height_and_balance() # Update height and␣
↪balance factor of k3_node's parent
        pass

    return root_node
```

[19]:
```
# k3
#   \
#   k1          =>      k2
#  /                   / \
# k2                  k1 k3

def RB_double_rotate_left(k3_node: RB_Node | None, root_node: RB_Node | None)␣
↪-> RB_Node | None:
    if PRINT_DEBUG:
        print(f"Double Left Rotate: k3_node={k3_node}, root_node={root_node}")
    k1_node = k3_node.rchild if k3_node is not None else None
    if k1_node is None or k3_node is None:
        return root_node

    if PRINT_DEBUG:
        print(f"Double Left Rotate: k1_node={k1_node}, k3_node={k3_node}")

    root_node = RB_right_rotate(k1_node, root_node)
    root_node = RB_left_rotate(k3_node, root_node)

    # k3_node.update_height_and_balance() # Update height and balance factor of␣
↪k3_node
    # k1_node.update_height_and_balance() # Update height and balance factor of␣
↪k1_node

    if k3_node.parent is not None:
        # k3_node.parent.update_height_and_balance() # Update height and␣
↪balance factor of k3_node's parent
```

```
        pass

    return root_node
```

[20]:
```
def RB_Insert_Fixup(root: RB_Node, Z_Node: RB_Node) -> RB_Node | None:
    global iterations
    while Z_Node.parent is not None and Z_Node.parent.color == '#F00':
        iterations += 1
        if Z_Node.parent == Z_Node.parent.parent.lchild:
            Y_Node = Z_Node.parent.parent.rchild
            if Y_Node is not None and Y_Node.color == '#F00':
                Z_Node.parent.color = '#000'
                Y_Node.color = '#000'
                Z_Node.parent.parent.color = '#F00'
                Z_Node = Z_Node.parent.parent
            else:
                if Z_Node == Z_Node.parent.rchild:
                    Z_Node = Z_Node.parent
                    root = RB_left_rotate(Z_Node, root) # type: ignore
                Z_Node.parent.color = '#000'
                Z_Node.parent.parent.color = '#F00'
                root = RB_right_rotate(Z_Node.parent.parent, root) # type:␣
 ↪ignore
        else:
            Y_Node = Z_Node.parent.parent.lchild
            if Y_Node is not None and Y_Node.color == '#F00':
                Z_Node.parent.color = '#000'
                Y_Node.color = '#000'
                Z_Node.parent.parent.color = '#F00'
                Z_Node = Z_Node.parent.parent
            else:
                if Z_Node == Z_Node.parent.lchild:
                    Z_Node = Z_Node.parent
                    root = RB_right_rotate(Z_Node, root) # type: ignore
                Z_Node.parent.color = '#000'
                Z_Node.parent.parent.color = '#F00'
                root = RB_left_rotate(Z_Node.parent.parent, root) # type: ignore
    root.color = '#000'
    return root

def RB_Insert(root: RB_Node | None, value: int) -> RB_Node | None:
    global iterations
    Z_Node = RB_Node(value)

    Y_NODE = None
    X_NODE = root
    while X_NODE is not None:
```

```
        iterations += 1
        Y_NODE = X_NODE
        if value < X_NODE.value:
            X_NODE = X_NODE.lchild
        else:
            X_NODE = X_NODE.rchild

    Z_Node.parent = Y_NODE

    if Y_NODE is None:
        root = Z_Node
    elif value < Y_NODE.value:
        Y_NODE.lchild = Z_Node
    else:
        Y_NODE.rchild = Z_Node

    Z_Node.lchild = None
    Z_Node.rchild = None
    # New Node's Color is already RED by default.

    root = RB_Insert_Fixup(root, Z_Node) # type: ignore
    return root
```

[21]:
```
def RB_inorder_traversal(root: RB_Node | None, out_file: TextIOWrapper | None)␣
↪-> None:
    if root is None:
        return
    RB_inorder_traversal(root.lchild, out_file = out_file)
    if out_file is not None:
        print(root.value, end = ', ', file = out_file)
    print(root.value, end = ', ')
    print(root)
    RB_inorder_traversal(root.rchild, out_file = out_file)

def RB_preorder_traversal(root: RB_Node | None, out_file: TextIOWrapper | None)␣
↪-> None:
    if root is None:
        return
    if out_file is not None:
        print(root.value, end = ', ', file = out_file)
    print(root.value, end = ', ')
    print(root)
    RB_preorder_traversal(root.lchild, out_file)
    RB_preorder_traversal(root.rchild, out_file)

def RB_postorder_traversal(root: RB_Node | None, out_file: TextIOWrapper |␣
↪None) -> None:
```

```
    if root is None:
        return
    RB_postorder_traversal(root.lchild, out_file)
    RB_postorder_traversal(root.rchild, out_file)
    if out_file is not None:
        print(root.value, end = ', ', file = out_file)
    print(root.value, end=', ')
    print(root)
```

[22]:
```
root0 = None
for value in arr0:
    root0 = RB_Insert(root0, value)
```

[23]:
```
RB_inorder_traversal(root0, out_file = None)
```

```
25, Node(value=25, color=#F00, lchild=None, rchild=None, parent=50)
50, Node(value=50, color=#000, lchild=25, rchild=None, parent=55)
55, Node(value=55, color=#000, lchild=50, rchild=67, parent=70)
67, Node(value=67, color=#000, lchild=None, rchild=69, parent=55)
69, Node(value=69, color=#F00, lchild=None, rchild=None, parent=67)
70, Node(value=70, color=#000, lchild=55, rchild=420, parent=None)
76, Node(value=76, color=#F00, lchild=None, rchild=None, parent=77)
77, Node(value=77, color=#000, lchild=76, rchild=None, parent=90)
90, Node(value=90, color=#F00, lchild=77, rchild=210, parent=420)
108, Node(value=108, color=#F00, lchild=None, rchild=None, parent=210)
210, Node(value=210, color=#000, lchild=108, rchild=348, parent=90)
348, Node(value=348, color=#F00, lchild=None, rchild=None, parent=210)
420, Node(value=420, color=#000, lchild=90, rchild=911, parent=70)
720, Node(value=720, color=#F00, lchild=None, rchild=None, parent=768)
768, Node(value=768, color=#000, lchild=720, rchild=None, parent=911)
911, Node(value=911, color=#F00, lchild=768, rchild=1080, parent=420)
1080, Node(value=1080, color=#000, lchild=None, rchild=None, parent=911)
```

Roll No.	Subject Code	Assignment No.
2650026	CS69101	Lab 5

2 Q3.

Implement delete(key) to remove a specified element from the tree and perform the required re-colouring and rotations/fix-up operations.

2.1 Deletion

[24]:
```
def RB_Search(root: RB_Node | None, key: int) -> RB_Node | None:
    global iterations
    X_NODE = root
    while X_NODE is not None:
```

```
        iterations += 1
        if key == X_NODE.value:
            return X_NODE
        X_NODE = X_NODE.lchild if key < X_NODE.value else X_NODE.rchild
    return None


def RB_Minimum(node: RB_Node) -> RB_Node:
    global iterations
    while node.lchild is not None:
        iterations += 1
        node = node.lchild
    return node


def RB_is_black(node: RB_Node | None) -> bool:
    # A missing (None) child is a black NIL leaf
    return node is None or node.color == '#000'


def RB_Transplant(root: RB_Node | None, u: RB_Node, v: RB_Node | None) ->␣
↪RB_Node | None:
    global iterations
    iterations += 1
    # CLRS RB-TRANSPLANT: replace the subtree rooted at u with the subtree␣
↪rooted at v
    if u.parent is None:
        root = v
        if v is not None:
            v.parent = None
    elif u is u.parent.lchild:
        u.parent.lchild = v   # setter also sets v.parent
    else:
        u.parent.rchild = v   # setter also sets v.parent
    return root
```

[25]:
```
def RB_Delete_Fixup(root: RB_Node | None, x: RB_Node | None, x_parent: RB_Node␣
↪| None) -> RB_Node | None:
    global iterations
    # x carries the extra black. x may be None (NIL), so its parent is tracked␣
↪separately.
    while x is not root and RB_is_black(x):
        if x_parent is None:
            break
        iterations += 1
        if x is x_parent.lchild:
            W_Node = x_parent.rchild
```

```
            if W_Node is not None and W_Node.color == '#F00':        # Case 1:␣
↪sibling red
                W_Node.color = '#000'
                x_parent.color = '#F00'
                root = RB_left_rotate(x_parent, root)
                W_Node = x_parent.rchild
            if RB_is_black(W_Node.lchild if W_Node is not None else None) and␣
↪RB_is_black(W_Node.rchild if W_Node is not None else None):
                # Case 2: both nephews black (a missing sibling is treated as␣
↪black)
                if W_Node is not None:
                    W_Node.color = '#F00'
                x = x_parent
                x_parent = x.parent
            else:
                if RB_is_black(W_Node.rchild if W_Node is not None else None): ␣
↪# Case 3: far nephew black
                    if W_Node is not None and W_Node.lchild is not None:
                        W_Node.lchild.color = '#000'
                    if W_Node is not None:
                        W_Node.color = '#F00'
                        root = RB_right_rotate(W_Node, root)
                    W_Node = x_parent.rchild
                if W_Node is not None:
                    W_Node.color = x_parent.color                          #␣
↪Case 4: far nephew red
                    x_parent.color = '#000'
                    if W_Node.rchild is not None:
                        W_Node.rchild.color = '#000'
                root = RB_left_rotate(x_parent, root)
                x = root
                x_parent = None
        else:                                                              # mirror␣
↪image
            W_Node = x_parent.lchild
            if W_Node is not None and W_Node.color == '#F00':
                W_Node.color = '#000'
                x_parent.color = '#F00'
                root = RB_right_rotate(x_parent, root)
                W_Node = x_parent.lchild
            if RB_is_black(W_Node.lchild if W_Node is not None else None) and␣
↪RB_is_black(W_Node.rchild if W_Node is not None else None):
                if W_Node is not None:
                    W_Node.color = '#F00'
                x = x_parent
                x_parent = x.parent
```

```
            else:
                if RB_is_black(W_Node.lchild if W_Node is not None else None):
                    if W_Node is not None and W_Node.rchild is not None:
                        W_Node.rchild.color = '#000'
                    if W_Node is not None:
                        W_Node.color = '#F00'
                        root = RB_left_rotate(W_Node, root)
                    W_Node = x_parent.lchild
                if W_Node is not None:
                    W_Node.color = x_parent.color
                    x_parent.color = '#000'
                    if W_Node.lchild is not None:
                        W_Node.lchild.color = '#000'
                root = RB_right_rotate(x_parent, root)
                x = root
                x_parent = None
    if x is not None:
        x.color = '#000'
    return root


def RB_Delete(root: RB_Node | None, key: int) -> RB_Node | None:
    Z_Node = RB_Search(root, key)
    if Z_Node is None:
        print(f"[WARN] Key {key} not found, nothing deleted")
        return root

    Y_Node = Z_Node
    Y_ORIG_COLOR = Y_Node.color

    if Z_Node.lchild is None:                    # 0 or 1 child (right)
        X_Node = Z_Node.rchild
        X_Parent = Z_Node.parent
        root = RB_Transplant(root, Z_Node, Z_Node.rchild)
    elif Z_Node.rchild is None:                  # 1 child (left)
        X_Node = Z_Node.lchild
        X_Parent = Z_Node.parent
        root = RB_Transplant(root, Z_Node, Z_Node.lchild)
    else:                                        # 2 children: use in-order␣
 ↪successor
        Y_Node = RB_Minimum(Z_Node.rchild)
        Y_ORIG_COLOR = Y_Node.color
        X_Node = Y_Node.rchild
        if Y_Node.parent is Z_Node:
            X_Parent = Y_Node
        else:
            X_Parent = Y_Node.parent
```

```
            root = RB_Transplant(root, Y_Node, Y_Node.rchild)
            Y_Node.rchild = Z_Node.rchild
        root = RB_Transplant(root, Z_Node, Y_Node)
        Y_Node.lchild = Z_Node.lchild
        Y_Node.color = Z_Node.color

    # Keep the NIL child's parent valid for the fix-up when it exists.
    if X_Node is not None:
        X_Parent = X_Node.parent

    # Detach the removed node
    Z_Node.lchild = None
    Z_Node.rchild = None
    Z_Node.parent = None

    if Y_ORIG_COLOR == '#000':                    # only a removed black node␣
↪breaks black-height
        root = RB_Delete_Fixup(root, X_Node, X_Parent)
    if root is not None:
        root.parent = None
    return root
```

[26]:
```
def RB_Verify(root: RB_Node | None) -> int:
    # Returns node count; raises AssertionError if any Red-Black property is␣
↪broken
    if root is None:
        return 0
    assert root.parent is None, "root has a parent"
    assert root.color == '#000', "root is not black"
    def rec(n, lo, hi):
        if n is None:
            return 0, 1
        assert lo <= n.value <= hi, f"BST order broken at {n.value}"
        assert n.color in ('#000', '#F00'), f"invalid color at {n.value}"
        if n.color == '#F00':
            assert RB_is_black(n.lchild) and RB_is_black(n.rchild), f"red-red␣
↪at {n.value}"
        for c in (n.lchild, n.rchild):
            assert c is None or c.parent is n, f"bad parent link under {n.
↪value}"
        cl, bl = rec(n.lchild, lo, n.value)
        cr, br = rec(n.rchild, n.value, hi)
        assert bl == br, f"black-height mismatch at {n.value}"
        return cl + cr + 1, bl + (1 if n.color == '#000' else 0)
    return rec(root, float('-inf'), float('inf'))[0]
```

```python
def RB_Height(root: RB_Node | None) -> int:
    if root is None:
        return 0
    return max(RB_Height(root.lchild), RB_Height(root.rchild)) + 1


def RB_Inorder_Values(root: RB_Node | None) -> list[int]:
    if root is None:
        return []
    return RB_Inorder_Values(root.lchild) + [root.value] +␣
 ↪RB_Inorder_Values(root.rchild)
```

[27]:
```python
root0 = RB_Delete(root0, 420)
RB_Verify(root0)

with open("./2650026_lab4_data/output.txt", "w") as out_file:
    RB_inorder_traversal(root0, out_file = out_file)
    out_file.close()
```

```
25, Node(value=25, color=#F00, lchild=None, rchild=None, parent=50)
50, Node(value=50, color=#000, lchild=25, rchild=None, parent=55)
55, Node(value=55, color=#000, lchild=50, rchild=67, parent=70)
67, Node(value=67, color=#000, lchild=None, rchild=69, parent=55)
69, Node(value=69, color=#F00, lchild=None, rchild=None, parent=67)
70, Node(value=70, color=#000, lchild=55, rchild=720, parent=None)
76, Node(value=76, color=#F00, lchild=None, rchild=None, parent=77)
77, Node(value=77, color=#000, lchild=76, rchild=None, parent=90)
90, Node(value=90, color=#F00, lchild=77, rchild=210, parent=720)
108, Node(value=108, color=#F00, lchild=None, rchild=None, parent=210)
210, Node(value=210, color=#000, lchild=108, rchild=348, parent=90)
348, Node(value=348, color=#F00, lchild=None, rchild=None, parent=210)
720, Node(value=720, color=#000, lchild=90, rchild=911, parent=70)
768, Node(value=768, color=#000, lchild=None, rchild=None, parent=911)
911, Node(value=911, color=#F00, lchild=768, rchild=1080, parent=720)
1080, Node(value=1080, color=#000, lchild=None, rchild=None, parent=911)
```

3 Q6

For different numbers of nodes, such as 100, 500, 1000, 5000, 10000 and 50000, measure and plot:

- Tree Height vs. Number of Nodes
- Insertion Time vs. Number of Nodes
- Deletion Time vs. Number of Nodes
- Search Time vs. Number of Nodes

The operation cost is measured using iteration/computation counts. The 50,000-node case is optional.

[28]:
```
# Use the same default sizes as the assignment; set INCLUDE_50000 = True to add the optional case.
SIZES = [100, 500, 1000, 5000, 10000]

random.seed(911)

def RB_Build_Measured(values: list[int]):
    global iterations
    root = None
    insert_counts = []
    for value in values:
        iterations = 0
        root = RB_Insert(root, value)
        insert_counts.append(iterations)
    return root, insert_counts


def RB_Search_Measured(root: RB_Node | None, key: int):
    global iterations
    iterations = 0
    node = RB_Search(root, key)
    return node, iterations


def RB_Delete_Measured(root: RB_Node | None, key: int):
    global iterations
    iterations = 0
    root = RB_Delete(root, key)
    return root, iterations


task6_rows = []
for n in SIZES:
    values = random.sample(range(1, n * 10 + 1), n)
    root, insert_counts = RB_Build_Measured(values)
    RB_Verify(root)
    height = RB_Height(root)

    search_keys = random.sample(values, min(100, n))
    search_counts = [RB_Search_Measured(root, key)[1] for key in search_keys]

    delete_keys = random.sample(values, min(100, n))
    delete_counts = []
    for key in delete_keys:
        root, count = RB_Delete_Measured(root, key)
        delete_counts.append(count)
    RB_Verify(root)
```

```
    task6_rows.append({
        "Nodes": n,
        "Height": height,
        "Average Insertion Iterations": sum(insert_counts) / len(insert_counts),
        "Average Deletion Iterations": sum(delete_counts) / len(delete_counts),
        "Average Search Iterations": sum(search_counts) / len(search_counts),
    })

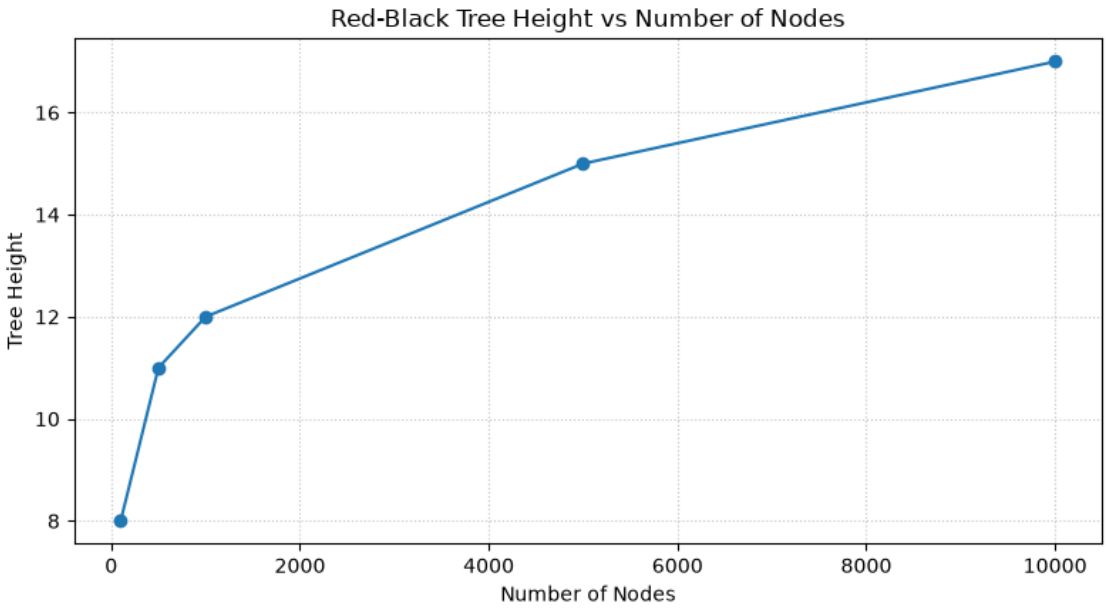
task6_df = pd.DataFrame(task6_rows)
print(task6_df)
```

```
   Nodes  Height  Average Insertion Iterations  Average Deletion Iterations  \
0    100       8                        6.5100                         7.23
1    500      11                        9.2680                        10.85
2   1000      12                       10.2530                        11.47
3   5000      15                       12.6540                        13.86
4  10000      17                       13.6435                        14.89

   Average Search Iterations
0                       5.95
1                       8.31
2                       9.08
3                      11.68
4                      12.33
```

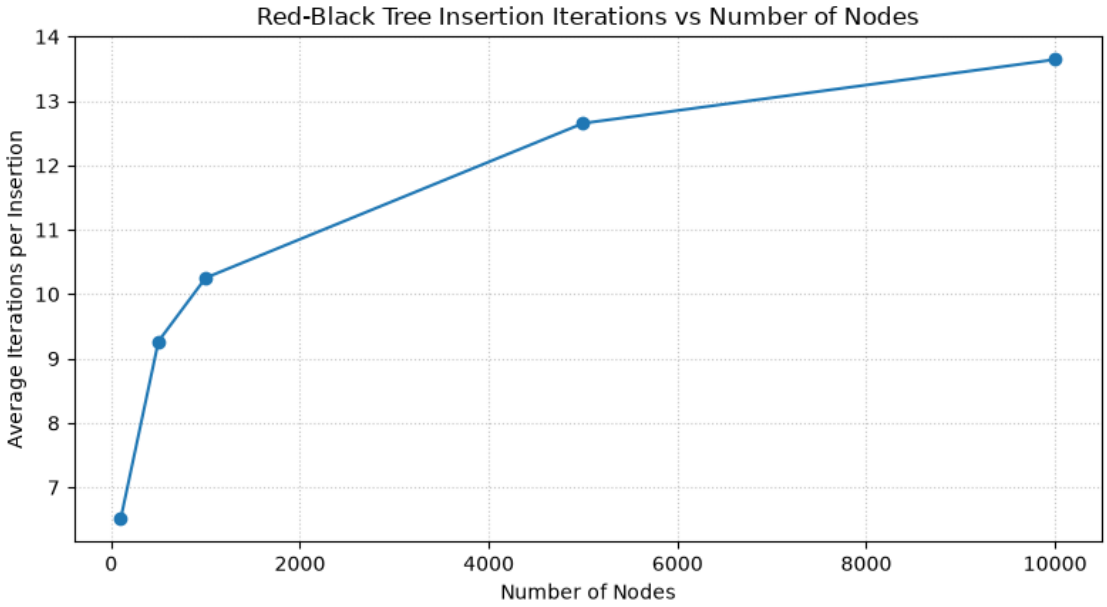
3.1 Height v/s Node Count

[30]:
```
plt.figure(figsize=(8, 4.5))
plt.plot(task6_df["Nodes"], task6_df["Height"], marker="o")
plt.xlabel("Number of Nodes")
plt.ylabel("Tree Height")
plt.title("Red-Black Tree Height vs Number of Nodes")
plt.grid(True, linestyle=":", alpha=0.7)
plt.tight_layout()
plt.savefig("./2650026_lab4_data/task6_height.png", dpi=150,␣
 ↪bbox_inches="tight")
plt.show()
```



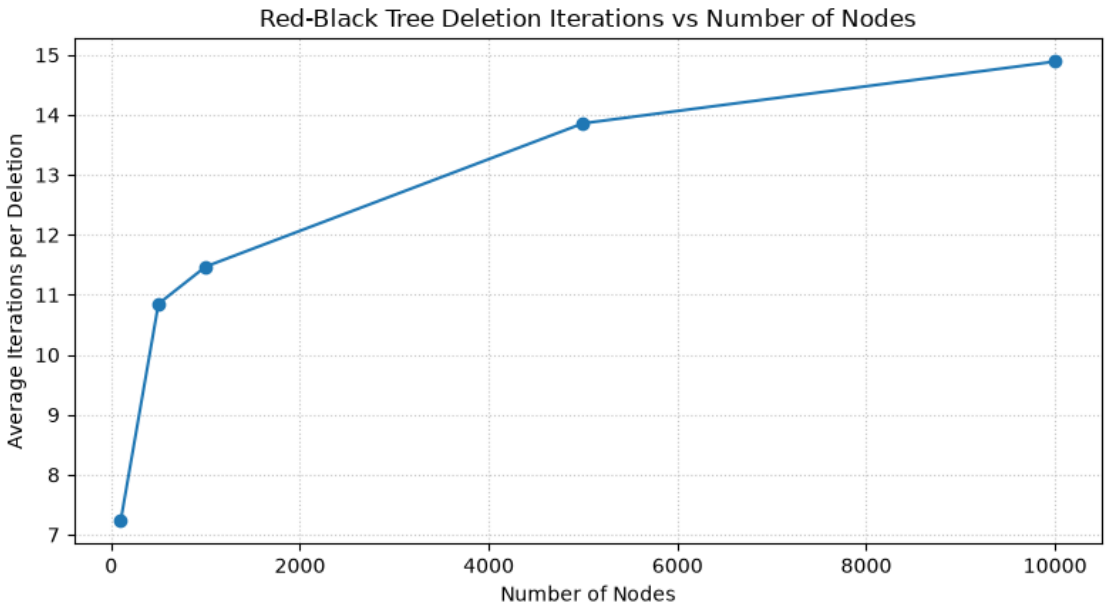
3.2 Iterations (Insertion) v/s Node Count

[31]:
```
plt.figure(figsize=(8, 4.5))
plt.plot(task6_df["Nodes"], task6_df["Average Insertion Iterations"],␣
 ↪marker="o")
plt.xlabel("Number of Nodes")
plt.ylabel("Average Iterations per Insertion")
plt.title("Red-Black Tree Insertion Iterations vs Number of Nodes")
plt.grid(True, linestyle=":", alpha=0.7)
plt.tight_layout()
plt.savefig("./2650026_lab4_data/task6_insertion.png", dpi=150,␣
 ↪bbox_inches="tight")
plt.show()
```



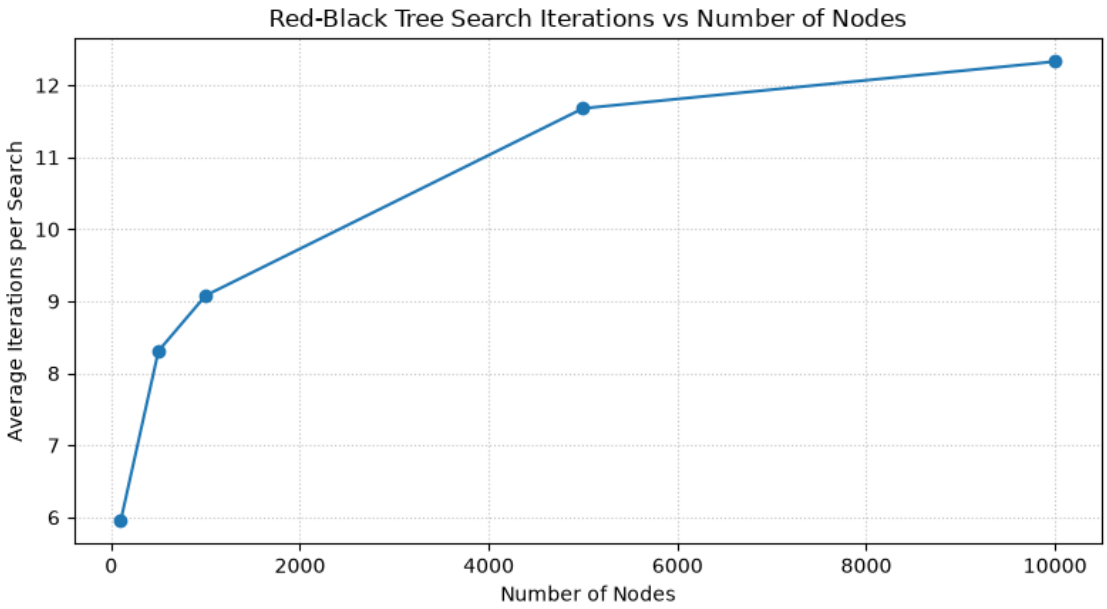
3.3 Iterations (Deletion) v/s Node Count

[32]:
```
plt.figure(figsize=(8, 4.5))
plt.plot(task6_df["Nodes"], task6_df["Average Deletion Iterations"], marker="o")
plt.xlabel("Number of Nodes")
plt.ylabel("Average Iterations per Deletion")
plt.title("Red-Black Tree Deletion Iterations vs Number of Nodes")
plt.grid(True, linestyle=":", alpha=0.7)
plt.tight_layout()
plt.savefig("./2650026_lab4_data/task6_deletion.png", dpi=150,␣
 ↪bbox_inches="tight")
plt.show()
```



3.4 Iterations (Search) v/s Node Count

[33]:
```
plt.figure(figsize=(8, 4.5))
plt.plot(task6_df["Nodes"], task6_df["Average Search Iterations"], marker="o")
plt.xlabel("Number of Nodes")
plt.ylabel("Average Iterations per Search")
plt.title("Red-Black Tree Search Iterations vs Number of Nodes")
plt.grid(True, linestyle=":", alpha=0.7)
plt.tight_layout()
plt.savefig("./2650026_lab4_data/task6_search.png", dpi=150,␣
 ↪bbox_inches="tight")
plt.show()
```



4 Q7

Implement an ordinary Binary Search Tree (BST) and experimentally compare its search time and height with the Red-Black Tree.

4.1 BST Tree: All Code

[34]:
```
class BST_Node:
    def __init__(self):
        self.lchild = None
        self.rchild = None
        self.value = None
        self.parent = None


def BST_Insert(value, root_node: BST_Node | None = None) -> BST_Node:
    assert value is not None, "Value cannot be None"
    y = None
    x = root_node
    global iterations
    while x is not None:
        iterations += 1
        y = x
        if value < x.value:
            x = x.lchild
        else:
```

```
            x = x.rchild
    new_node = BST_Node()
    new_node.value = value
    new_node.parent = y
    if y is None:
        root_node = new_node
    elif value < y.value:
        y.lchild = new_node
    else:
        y.rchild = new_node
    return root_node


def BST_Search(value, root_node: BST_Node | None = None) -> BST_Node | None:
    global iterations
    if root_node is None:
        return None
    iterations += 1
    if value == root_node.value:
        return root_node
    elif value < root_node.value:
        return BST_Search(value, root_node.lchild)
    else:
        return BST_Search(value, root_node.rchild)


def BST_minimum(root_node: BST_Node | None = None) -> BST_Node | None:
    global iterations
    if root_node is None:
        return None
    while root_node.lchild is not None:
        iterations += 1
        root_node = root_node.lchild
    return root_node


def BST_Transplant(root_node: BST_Node | None = None, target_node: BST_Node |␣
 ↪None = None, replacement_node: BST_Node | None = None) -> BST_Node | None:
    global iterations
    iterations += 1
    if root_node is None:
        return None
    if target_node is None:
        return root_node
    if target_node.parent is None:
        root_node = replacement_node
    elif target_node == target_node.parent.lchild:
```

```
        target_node.parent.lchild = replacement_node
    else:
        target_node.parent.rchild = replacement_node
    if replacement_node is not None:
        replacement_node.parent = target_node.parent
    return root_node


def BST_Delete(root_node: BST_Node | None = None, target_node: BST_Node | None␣
 ↪= None) -> BST_Node | None:
    global iterations
    iterations += 1
    if root_node is None:
        return None
    if target_node is None:
        return root_node
    if target_node.lchild is None:
        root_node = BST_Transplant(root_node, target_node, target_node.rchild)
    elif target_node.rchild is None:
        root_node = BST_Transplant(root_node, target_node, target_node.lchild)
    else:
        successor_node = BST_minimum(target_node.rchild)
        if successor_node.parent != target_node:
            root_node = BST_Transplant(root_node, successor_node,␣
 ↪successor_node.rchild)
            successor_node.rchild = target_node.rchild
            successor_node.rchild.parent = successor_node
        root_node = BST_Transplant(root_node, target_node, successor_node)
        successor_node.lchild = target_node.lchild
        successor_node.lchild.parent = successor_node
    return root_node


def BST_Height(root_node: BST_Node | None = None) -> int:
    global iterations
    if root_node is None:
        return 0
    iterations += 1
    left_height = BST_Height(root_node.lchild)
    right_height = BST_Height(root_node.rchild)
    return max(left_height, right_height) + 1


def BST_Build_Measured(values: list[int]):
    global iterations
    root = None
    insert_counts = []
```

```
    for value in values:
        iterations = 0
        root = BST_Insert(value, root)
        insert_counts.append(iterations)
    return root, insert_counts


def BST_Search_Measured(root: BST_Node | None, key: int):
    global iterations
    iterations = 0
    node = BST_Search(key, root)
    return node, iterations


def BST_Delete_Measured(root: BST_Node | None, key: int):
    global iterations
    iterations = 0
    node = BST_Search(key, root)  # Include the search required to locate the␣
 ↪target
    root = BST_Delete(root, node)
    return root, iterations


def BST_Inorder_Values(root: BST_Node | None) -> list[int]:
    if root is None:
        return []
    return BST_Inorder_Values(root.lchild) + [root.value] +␣
 ↪BST_Inorder_Values(root.rchild)
```

[35]:
```
task7_rows = []
for n in SIZES:
    values = random.sample(range(1, n * 10 + 1), n)
    bst_root, _ = BST_Build_Measured(values)
    rb_root, _ = RB_Build_Measured(values)
    RB_Verify(rb_root)

    # Height calculation is performed separately from operation iteration␣
 ↪counts.
    bst_height = BST_Height(bst_root)
    rb_height = RB_Height(rb_root)
    search_keys = random.sample(values, min(100, n))
    bst_search = [BST_Search_Measured(bst_root, key)[1] for key in search_keys]
    rb_search = [RB_Search_Measured(rb_root, key)[1] for key in search_keys]

    task7_rows.append({
        "Nodes": n,
        "BST Height": bst_height,
```

```
        "Red-Black Tree Height": rb_height,
        "BST Average Search Iterations": sum(bst_search) / len(bst_search),
        "Red-Black Tree Average Search Iterations": sum(rb_search) /␣
 ↪len(rb_search),
    })

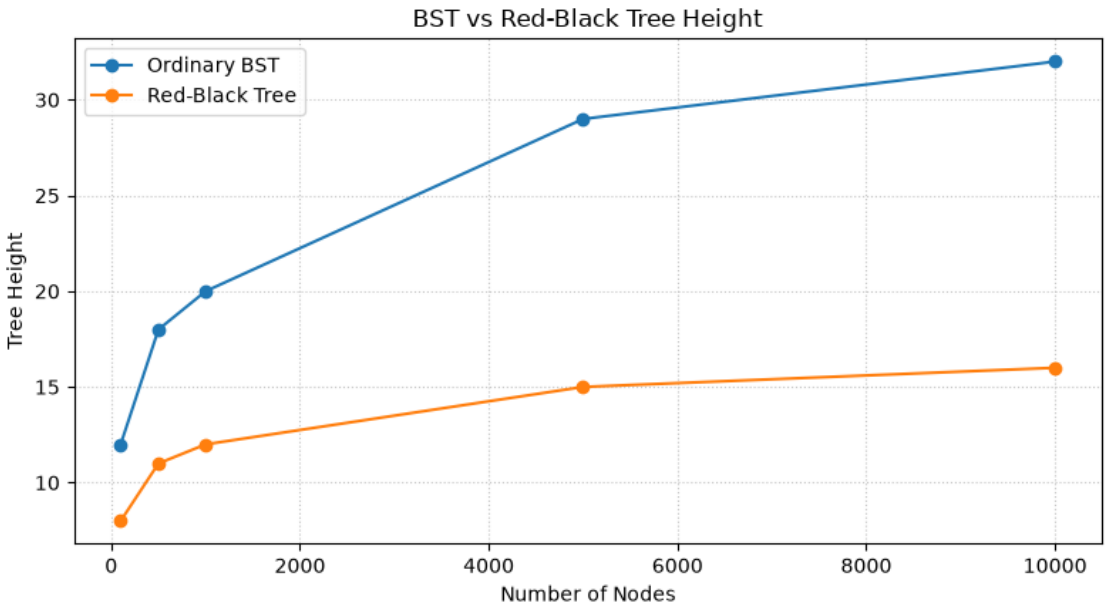
task7_df = pd.DataFrame(task7_rows)
task7_df
```

[35]:

	Nodes	BST Height	Red-Black Tree Height	BST Average Search Iterations	Red-Black Tree Average Search Iterations
0	100	12	8	6.64	5.96
1	500	18	11	10.46	8.20
2	1000	20	12	11.35	9.18
3	5000	29	15	15.33	11.66
4	10000	32	16	16.35	12.55

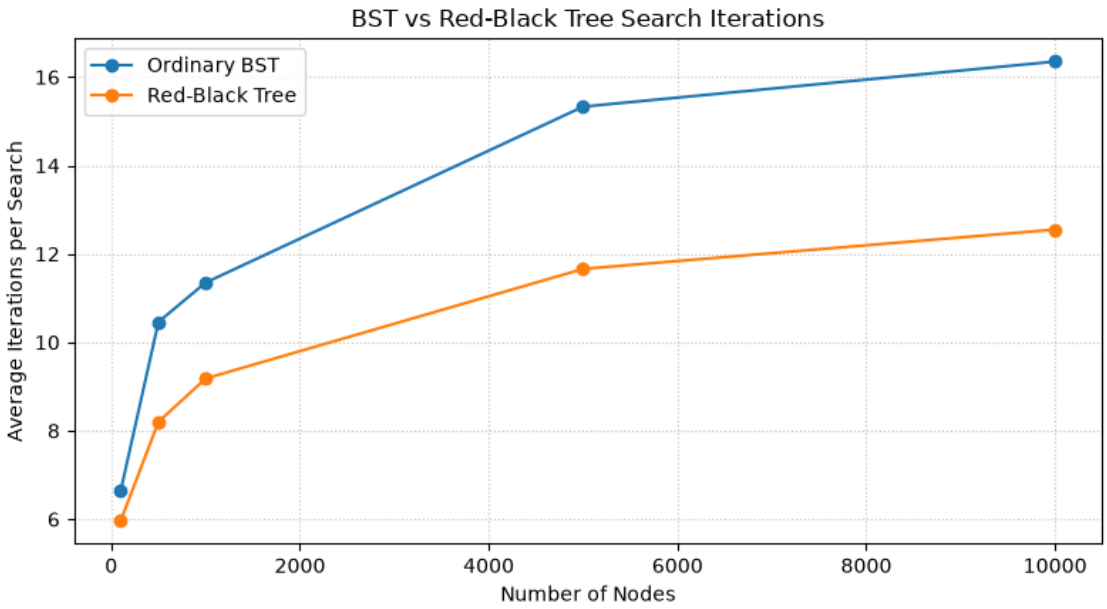
4.2 Height Comparison

[37]:
```
plt.figure(figsize=(8, 4.5))
plt.plot(task7_df["Nodes"], task7_df["BST Height"], marker="o", label="Ordinary␣
 ↪BST")
plt.plot(task7_df["Nodes"], task7_df["Red-Black Tree Height"], marker="o",␣
 ↪label="Red-Black Tree")
plt.xlabel("Number of Nodes")
plt.ylabel("Tree Height")
plt.title("BST vs Red-Black Tree Height")
plt.legend()
plt.grid(True, linestyle=":", alpha=0.7)
plt.tight_layout()
plt.savefig("./2650026_lab4_data/task7_height.png", dpi=150,␣
 ↪bbox_inches="tight")
plt.show()
```



4.3 Search Comparison

[38]:
```
plt.figure(figsize=(8, 4.5))
plt.plot(task7_df["Nodes"], task7_df["BST Average Search Iterations"],␣
 ↪marker="o", label="Ordinary BST")
plt.plot(task7_df["Nodes"], task7_df["Red-Black Tree Average Search␣
 ↪Iterations"], marker="o", label="Red-Black Tree")
plt.xlabel("Number of Nodes")
plt.ylabel("Average Iterations per Search")
plt.title("BST vs Red-Black Tree Search Iterations")
plt.legend()
plt.grid(True, linestyle=":", alpha=0.7)
plt.tight_layout()
plt.savefig("./2650026_lab4_data/task7_search.png", dpi=150,␣
 ↪bbox_inches="tight")
plt.show()
```



5 Q8

Experiment using random, sorted, and reverse-sorted input data and provide a brief analysis of the results.

For each size, the same unique values are tested in all three input orders. Insertion work is collected while building each tree, successful searches are measured before deletion, and up to 100 keys are then deleted. The graphs and measurements are analysed below.

[39]:
```
task8_rows = []
for n in SIZES:
    random_values = random.sample(range(1, n * 10 + 1), n)
    unique_values = sorted(random_values)
    input_orders = {
        "Random": random_values,
        "Sorted": unique_values,
        "Reverse Sorted": list(reversed(unique_values)),
    }
    search_keys = random.sample(unique_values, min(100, n))
    delete_keys = random.sample(unique_values, min(100, n))

    for order_name, values in input_orders.items():
        for tree_name in ["BST", "Red-Black Tree"]:
            if tree_name == "BST":
                root, insert_counts = BST_Build_Measured(values)
                height = BST_Height(root)
```

```
                search_counts = [BST_Search_Measured(root, key)[1] for key in␣
↪search_keys]
                delete_counts = []
                for key in delete_keys:
                    root, count = BST_Delete_Measured(root, key)
                    delete_counts.append(count)
                inorder_values = BST_Inorder_Values(root)
                assert inorder_values == sorted(inorder_values), "BST ordering␣
↪broken after deletion"
            else:
                root, insert_counts = RB_Build_Measured(values)
                RB_Verify(root)
                height = RB_Height(root)
                search_counts = [RB_Search_Measured(root, key)[1] for key in␣
↪search_keys]
                delete_counts = []
                for key in delete_keys:
                    root, count = RB_Delete_Measured(root, key)
                    delete_counts.append(count)
                RB_Verify(root)
                inorder_values = RB_Inorder_Values(root)
                assert inorder_values == sorted(inorder_values), "Red-Black␣
↪Tree ordering broken after deletion"

            task8_rows.append({
                "Nodes": n,
                "Input Order": order_name,
                "Tree": tree_name,
                "Height": height,
                "Average Insertion Iterations": sum(insert_counts) /␣
↪len(insert_counts),
                "Average Deletion Iterations": sum(delete_counts) /␣
↪len(delete_counts),
                "Average Search Iterations": sum(search_counts) /␣
↪len(search_counts),
                "Nodes Remaining": len(inorder_values),
            })

task8_df = pd.DataFrame(task8_rows)
task8_df
```

[39]:

	Nodes	Input Order	Tree	Height	\
0	100	Random	BST	18	
1	100	Random	Red-Black Tree	8	
2	100	Sorted	BST	100	
3	100	Sorted	Red-Black Tree	11	

4	100	Reverse Sorted	BST	100	
5	100	Reverse Sorted	Red-Black Tree	11	
6	500	Random	BST	17	
7	500	Random	Red-Black Tree	11	
8	500	Sorted	BST	500	
9	500	Sorted	Red-Black Tree	15	
10	500	Reverse Sorted	BST	500	
11	500	Reverse Sorted	Red-Black Tree	15	
12	1000	Random	BST	23	
13	1000	Random	Red-Black Tree	12	
14	1000	Sorted	BST	1000	
15	1000	Sorted	Red-Black Tree	17	
16	1000	Reverse Sorted	BST	1000	
17	1000	Reverse Sorted	Red-Black Tree	17	
18	5000	Random	BST	30	
19	5000	Random	Red-Black Tree	15	
20	5000	Sorted	BST	5000	
21	5000	Sorted	Red-Black Tree	22	
22	5000	Reverse Sorted	BST	5000	
23	5000	Reverse Sorted	Red-Black Tree	22	
24	10000	Random	BST	31	
25	10000	Random	Red-Black Tree	16	
26	10000	Sorted	BST	10000	
27	10000	Sorted	Red-Black Tree	24	
28	10000	Reverse Sorted	BST	10000	
29	10000	Reverse Sorted	Red-Black Tree	24	

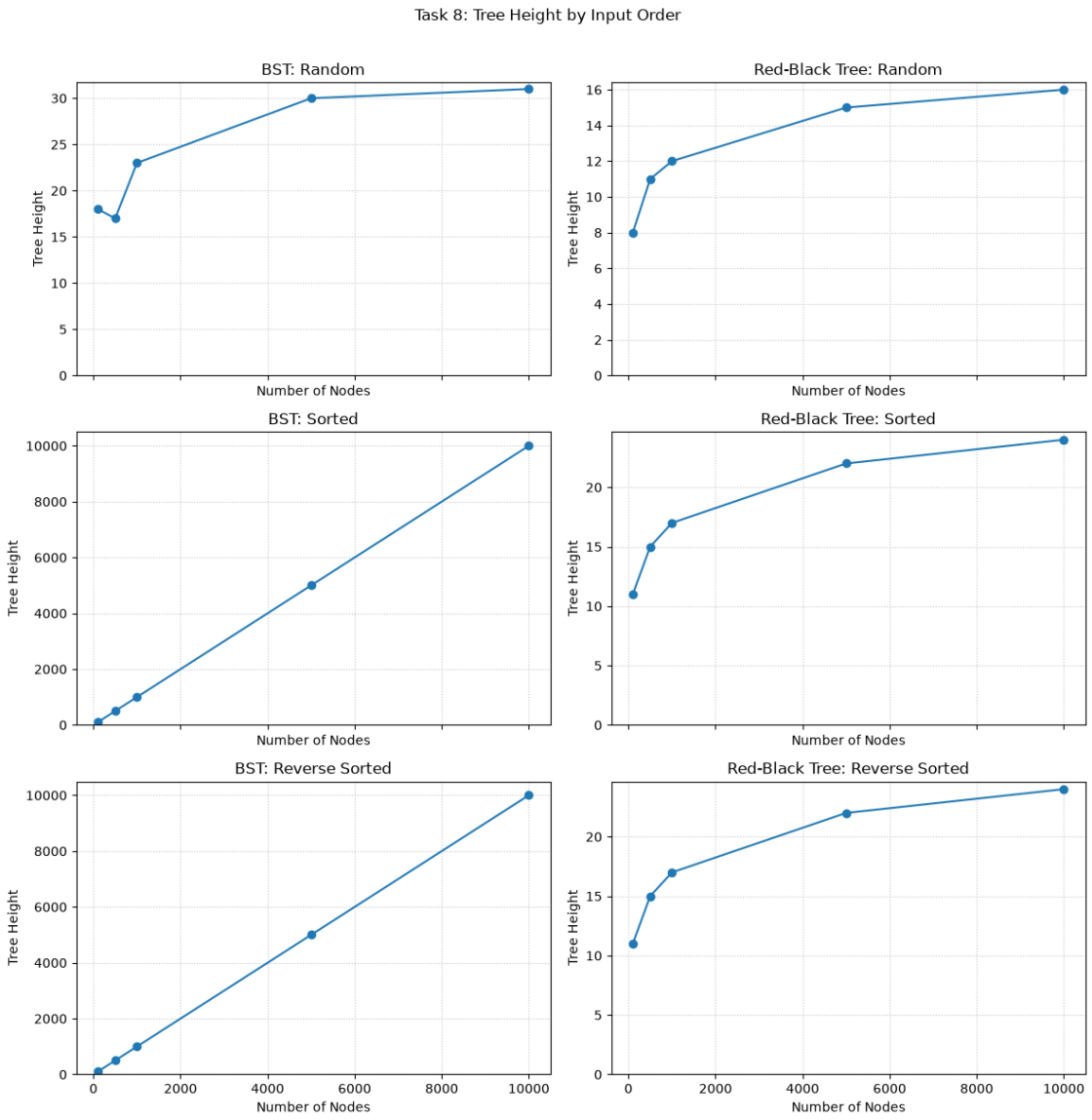
	Average Insertion Iterations	Average Deletion Iterations	\
0	7.1900	8.83	
1	6.8100	7.11	
2	49.5000	30.99	
3	10.7400	7.72	
4	49.5000	24.51	
5	10.7400	7.48	
6	9.0520	12.65	
7	9.3120	10.65	
8	249.5000	211.46	
9	15.4140	11.53	
10	249.5000	244.04	
11	15.4140	11.60	
12	11.1820	14.77	
13	10.2440	11.42	
14	499.5000	440.23	
15	17.4250	13.52	
16	499.5000	515.27	
17	17.4250	13.43	
18	13.5454	16.32	

19	12.6274	13.87
20	2499.5000	2303.68
21	22.1288	17.67
22	2499.5000	2651.82
23	22.1288	17.80
24	14.9784	19.03
25	13.6160	14.67
26	4999.5000	4919.90
27	24.1306	19.89
28	4999.5000	5035.60
29	24.1306	19.95

	Average Search Iterations	Nodes Remaining
0	8.19	0
1	5.86	0
2	50.50	0
3	6.20	0
4	50.50	0
5	6.20	0
6	10.17	400
7	8.39	400
8	250.72	400
9	8.39	400
10	250.28	400
11	8.51	400
12	12.15	900
13	8.97	900
14	556.44	900
15	9.55	900
16	444.56	900
17	9.23	900
18	14.06	4900
19	11.68	4900
20	2543.41	4900
21	12.01	4900
22	2457.59	4900
23	11.92	4900
24	16.13	9900
25	12.49	9900
26	5212.72	9900
27	12.92	9900
28	4788.28	9900
29	13.00	9900

5.1 Height Comparison

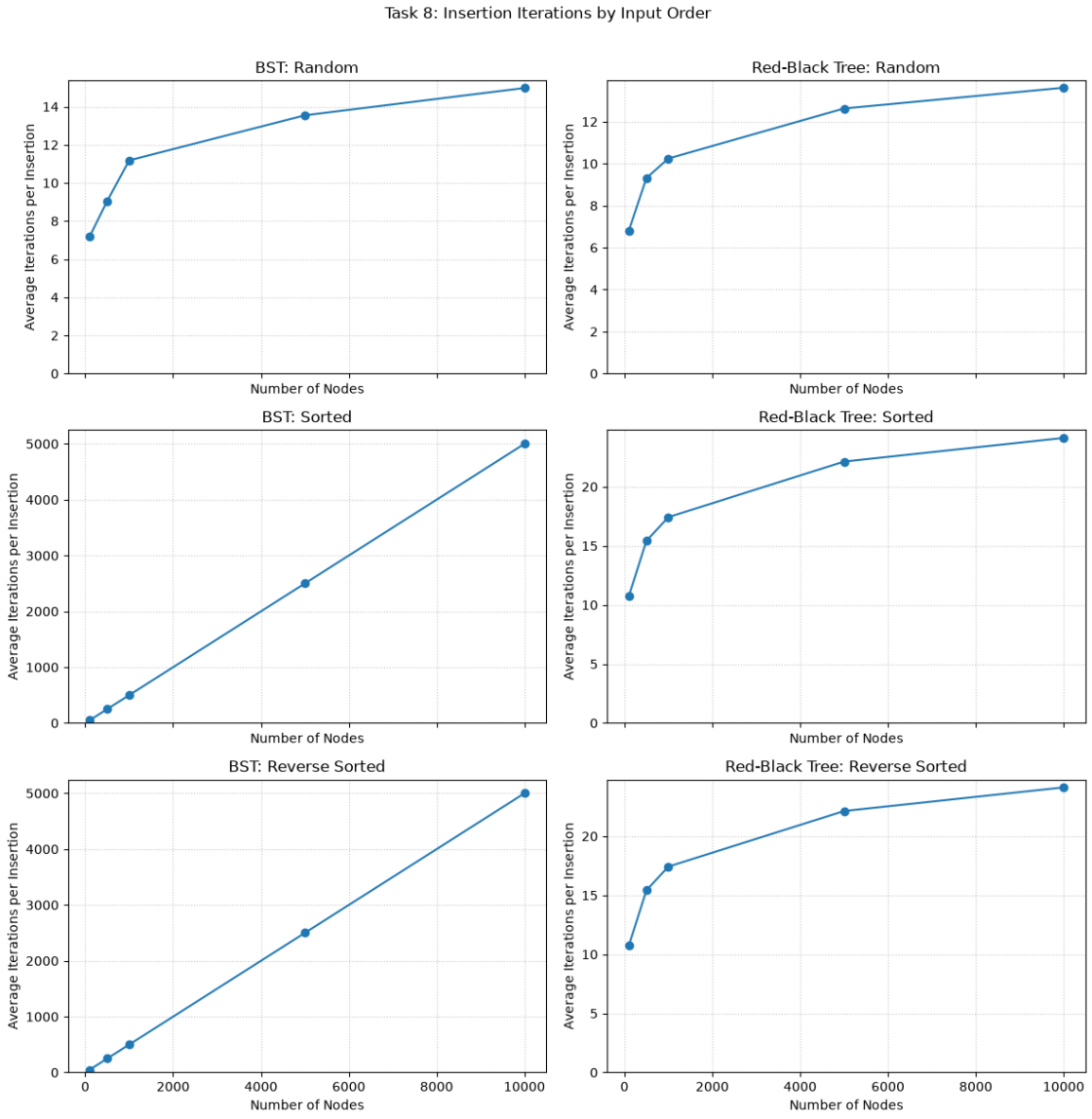
[40]:
```
fig, axes = plt.subplots(3, 2, figsize=(12, 12), sharex=True)
for row, order_name in enumerate(["Random", "Sorted", "Reverse Sorted"]):
    for col_idx, tree_name in enumerate(["BST", "Red-Black Tree"]):
        ax = axes[row, col_idx]
        subset = task8_df[(task8_df["Input Order"] == order_name) &␣
 ↪(task8_df["Tree"] == tree_name)]
        ax.plot(subset["Nodes"], subset["Height"], marker="o")
        ax.set_title(f"{tree_name}: {order_name}")
        ax.set_xlabel("Number of Nodes")
        ax.set_ylabel("Tree Height")
        ax.set_ylim(bottom=0)
        ax.grid(True, linestyle=":", alpha=0.7)
fig.suptitle("Task 8: Tree Height by Input Order", y=1.01)
plt.tight_layout()
plt.savefig("./2650026_lab4_data/task8_height.png", dpi=150,␣
 ↪bbox_inches="tight")
plt.show()
```



5.2 Insertion Comparison

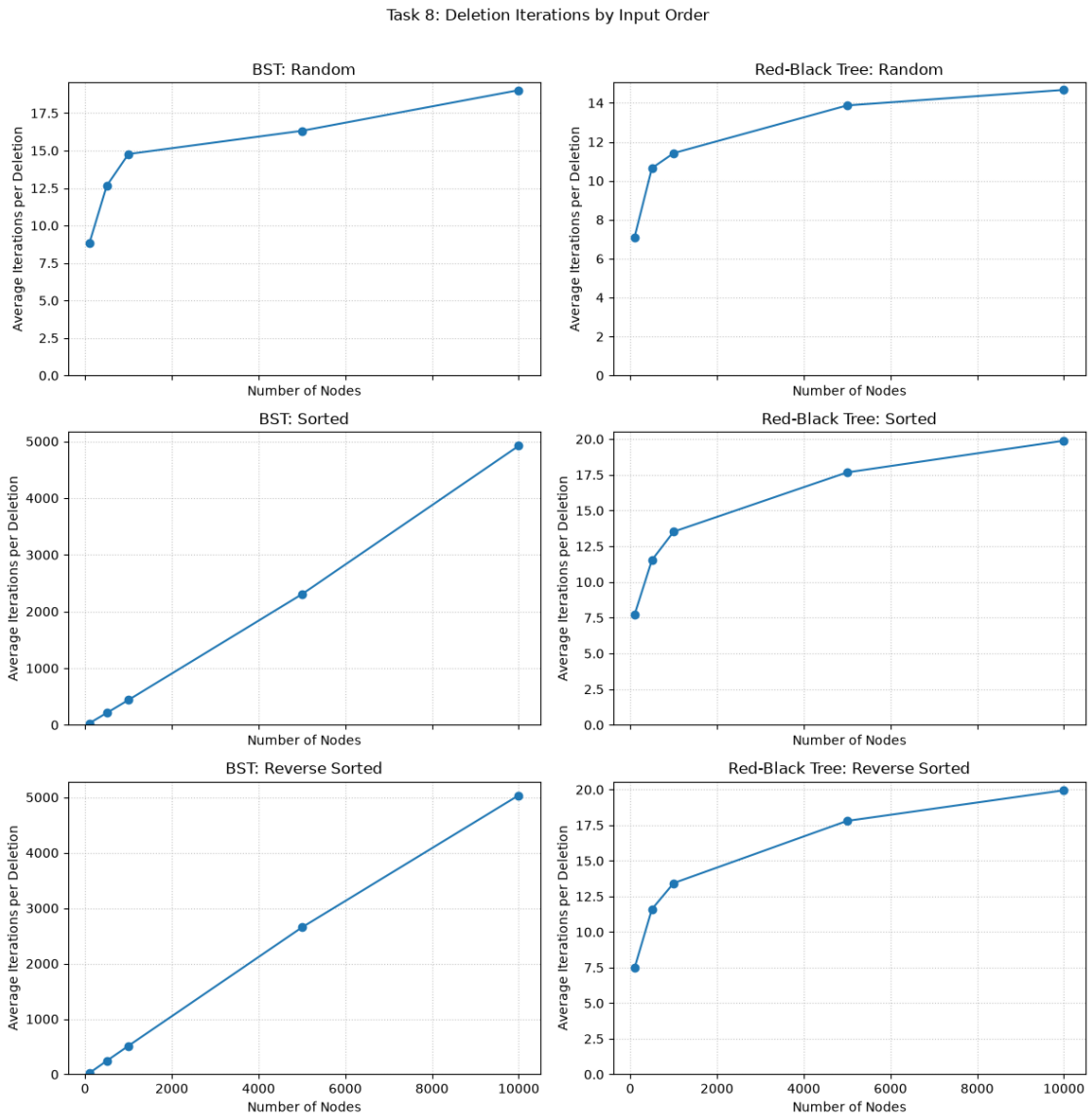
[41]:
```
fig, axes = plt.subplots(3, 2, figsize=(12, 12), sharex=True)
for row, order_name in enumerate(["Random", "Sorted", "Reverse Sorted"]):
    for col_idx, tree_name in enumerate(["BST", "Red-Black Tree"]):
        ax = axes[row, col_idx]
        subset = task8_df[(task8_df["Input Order"] == order_name) &␣
↪(task8_df["Tree"] == tree_name)]
        ax.plot(subset["Nodes"], subset["Average Insertion Iterations"],␣
↪marker="o")
        ax.set_title(f"{tree_name}: {order_name}")
        ax.set_xlabel("Number of Nodes")
```

```
        ax.set_ylabel("Average Iterations per Insertion")
        ax.set_ylim(bottom=0)
        ax.grid(True, linestyle=":", alpha=0.7)
fig.suptitle("Task 8: Insertion Iterations by Input Order", y=1.01)
plt.tight_layout()
plt.savefig("./2650026_lab4_data/task8_insertion.png", dpi=150,␣
  ↪bbox_inches="tight")
plt.show()
```



5.3 Deletion Comparison

[42]:
```
fig, axes = plt.subplots(3, 2, figsize=(12, 12), sharex=True)
for row, order_name in enumerate(["Random", "Sorted", "Reverse Sorted"]):
    for col_idx, tree_name in enumerate(["BST", "Red-Black Tree"]):
        ax = axes[row, col_idx]
        subset = task8_df[(task8_df["Input Order"] == order_name) &␣
 ↪(task8_df["Tree"] == tree_name)]
        ax.plot(subset["Nodes"], subset["Average Deletion Iterations"],␣
 ↪marker="o")
        ax.set_title(f"{tree_name}: {order_name}")
        ax.set_xlabel("Number of Nodes")
        ax.set_ylabel("Average Iterations per Deletion")
        ax.set_ylim(bottom=0)
        ax.grid(True, linestyle=":", alpha=0.7)
fig.suptitle("Task 8: Deletion Iterations by Input Order", y=1.01)
plt.tight_layout()
plt.savefig("./2650026_lab4_data/task8_deletion.png", dpi=150,␣
 ↪bbox_inches="tight")
plt.show()
```



5.4 Search Comparison

[43]:
```
fig, axes = plt.subplots(3, 2, figsize=(12, 12), sharex=True)
for row, order_name in enumerate(["Random", "Sorted", "Reverse Sorted"]):
    for col_idx, tree_name in enumerate(["BST", "Red-Black Tree"]):
        ax = axes[row, col_idx]
        subset = task8_df[(task8_df["Input Order"] == order_name) &␣
↪(task8_df["Tree"] == tree_name)]
        ax.plot(subset["Nodes"], subset["Average Search Iterations"],␣
↪marker="o")
        ax.set_title(f"{tree_name}: {order_name}")
        ax.set_xlabel("Number of Nodes")
```

```
        ax.set_ylabel("Average Iterations per Search")
        ax.set_ylim(bottom=0)
        ax.grid(True, linestyle=":", alpha=0.7)
fig.suptitle("Task 8: Search Iterations by Input Order", y=1.01)
plt.tight_layout()
plt.savefig("./2650026_lab4_data/task8_search.png", dpi=150,␣
  ↪bbox_inches="tight")
plt.show()
```

